

Neumann János Egyetem

Dinamikus Moziműsor Nyilvántartó Rendszer

Palotai Tamás, Horvát Evelin

2026.01.15.

Fejlesztői környezet: Php storm, Xamp

Alkalmazott nyelvek és technológiák: PHP, HTML, CSS, JavaScript (Vanilla JS, OOJS), React (JSX), MySQL, JSON, Axios, Babel.

Tartalomjegyzék

Feladat ismertetése:.....	2
Előfeltételek	3
Feladatmegosztás (Projektmunka).....	4
Deklarációk:	5
Adatbázis	8
Használati dokumentáció Funkcionális bemutatás (Használati útmutató).....	10
5 Központi API (api.php)	10
A teljes program kódja magyarázatokkal:	19

Feladat ismertetése:

A jelenlegi rendszerben mit kell fejleszteni:

A projekt keretében egy webalapú, többretegű (kliens-szerver architektúrájú) mozi- és műsornyilvántartó alkalmazás kifejlesztése a feladat. A háttérrendszerben egy központosított, PHP alapú REST API-t kell létrehozni, amely MySQL adatbázisból szolgálja ki az adatokat. A kliensoldalon pedig ennek a végpontnak a használatát kell demonstrálni, megvalósítva a teljes CRUD (Create, Read, Update, Delete) funkcionalitást többféle modern webes technológiával, úgymint: aszinkron Vanilla JavaScript, Fetch API, Axios könyvtár, valamint React.js alapú (Single Page Application) megközelítésekkel. Ezen felül a projektnek tartalmaznia kell az Objektumorientált JavaScript (OOJS) gyakorlati alkalmazását is egy grafikus elemen (téglaalap rajzoló).

Mi a problémája:

A hagyományos, statikus weboldalak vagy asztali nyilvántartások nem teszik lehetővé a moziadatok (nevek, címek, filmek) valós idejű, oldal újra töltés nélküli (aszinkron) kezelését és frissítését. Egy modern felhasználói felületnél elvárás a gyors, dinamikus adatcsere. Továbbá oktatási és fejlesztési szempontból gyakran probléma a megfelelő technológia kiválasztása, ezért szükség van egy olyan egységes felületre, amely gyakorlati példákon keresztül, egymás mellett hasonlítja össze a különböző frontend adatelérési módszereket ugyanarra az adatbázisra támaszkodva.

Miben tud segítségünkre lenni a program:

Az elkészült program egy központosított, rendkívül gyors és reszponzív (felhasználóbarát) webes felületet biztosít a mozik adatainak adminisztrációjára. Mivel az adatkérés és -mentés aszinkron módon történik a háttérben, a felhasználónak nem kell várakoznia az oldal újra töltésére egy-egy új mozi felvitele vagy törlése után. Szoftverfejlesztési szempontból a program kiváló demonstrációs eszköz: egységesen mutatja be a modern webfejlesztés lépcsőfokait a nyers JavaScripttől kezdve a komponensalapú React keretrendszerig, jelentősen segítve ezzel a különböző technológiák megértését.

Előfeltételek

Szerveroldal: XAMPP (Apache, MySQL/MariaDB), PHP.

Adatbázis kezelés: PDO (PHP Data Objects) a biztonságos, SQL injection elleni védelem érdekében. Frontend: HTML5, CSS3 (Flexbox elrendezés), Vanilla JavaScript (ES6+).

Keretrendszerek: React.js.

Könyvtárak: Axios (HTTP kérésekhez), Babel (JSX fordításhoz).

Kliens: Modern webböngésző (pl. Chrome, Firefox, Edge) a JavaScript (ES6), a Fetch API és a React kódok megfelelő futtatásához.

Github: https://github.com/evelinhrvt/Webprog_1_eloadas_beadando

Weboldal: <http://bead.mozibeadandopmb6zmaj4b6q.nhely.hu>

Feladatmegosztás (Projektmunka)

A fejlesztési folyamat során kiemelt figyelmet fordítottunk a csoportmunkára, megfelelően a kiírás 10. pontjának. A kód bázisát strukturáltan osztottuk meg: míg a feltöltéseket a hatékonyság érdekében közös munkaállomásról vagy technikai egyeztetést követően adminisztráltuk a GitHubra, addig az egyes funkcionális egységeket a fenti munkamegosztás szerint külön-külön fejlesztettük le.

Palotai Tamás (PMB6ZM):

Adatbázis és Backend: A MySQL adatbázis megtervezése, a kapcsolat.php és a központi api.php (REST API) kifejlesztése.

Klasszikus Frontend: Az alapvető stíluslap (style.css) és a navigációs logika (menu.js) elkészítése.

Native JS Implementációk: A javascript.html (tömbalapú CRUD) és a fetchapi.html (szerveroldali Fetch API) modulok programozása.

Horvát Evelin (AJ4B6Q):

React Keretrendszer: A React környezet kialakítása és a React.html (React CRUD) modul megvalósítása.

Modern Adatkezelés: Az axios.html (React + Axios alapú CRUD) felület és logika kidolgozása.

SPA és OOJS: Az Egyoldalas Alkalmazás (spa.html) számláló és színváltó komponenseinek, valamint az objektumorientált grafikai modulnak (oojs.html) a fejlesztése.

Deklarációk:

kapcsolat.php

Változó neve	Programban mire használjuk
\$port	Az adatbázis szerver portját tárolja (3306).
\$host	Az adatbázis szerver címét határozza meg (localhost).
\$dbname	A kapcsolódáshoz szükséges adatbázis neve (mozimisor).
\$user	Az adatbázis-felhasználó neve (mozis).
\$pass	Az adatbázis-felhasználó jelszava (mozis).
\$charset	A karakterkódolást állítja be (utf8mb4).
\$dsn	Data Source Name: a PDO számára átadott kapcsolódási paraméterlánc.
\$options	Tömb, amely a PDO objektum beállításait (hibakezelés, lekérési mód) tartalmazza.
\$pdo	Maga az aktív, felépített PDO adatbázis-kapcsolat objektuma. Ezt használják a további fájlok a lekérdezésekhez.

api.php

\$method	A beérkező HTTP kérés típusát tárolja (GET, POST, DELETE). Ez alapján dől el, melyik ág fut le.
\$stmt	(PDOStatement) Az előkészített és végrehajtott SQL lekérdezést tartalmazó objektum.
\$mozik	A GET kérés esetén az adatbázisból kinyert adatok (mozik) asszociatív tömbje.
\$data	A kliens oldalról (POST vagy DELETE kérés) érkező, JSON formátumból dekódolt PHP objektum.

menu.js

headerEsNav	Konstans szöveges változó (string), amely a fejléc (<header>) és a menüsor (<nav>) HTML kódját tárolja.
footer	Konstans szöveges változó, amely az oldal alján megjelenő lábléc (<footer>) HTML kódját tartalmazza.

javascript.html és fetchapi.html

API_URL	Konstans változó, az API végpont relatív útvonalát tárolja (api.php).
mozik	Tömb, a backendről letöltött mozi-objektumokat tárolja a memóriában.
betoltMozik() / betoltes()	Aszinkron függvény, amely a Fetch API segítségével GET kérést indít, és betölti az adatokat a felületre.
megjelenitMozik()	DOM-manipulációs függvény, az adatokat a HTML listában (ul) rendereli ki.
hozzaadMozi() / mentes()	Aszinkron függvény, kiolvassa az input mezők értékeit és POST kéréssel az API-hoz küldi mentésre.
torolMozi(id) / torles(id)	Aszinkron függvény, egy konkrét mozi azonosítóját (id) elküldi DELETE kéréssel az API-nak.

react.html és axios.php

mozik / setMozik	React State: A lekérdezett mozik listáját tárolja és frissíti a komponensen belül.
nev / setNev	React State: A "Mozi neve" űrlapmező aktuális értékét kezeli.
cim / setCim	React State: A "Címe" űrlapmező aktuális értékét kezeli.
hiba / setHiba	(Axios példában) React State: A backenddel való kommunikáció során fellépő hibaüzenetek tárolására és megjelenítésére szolgál.
betoltMozik() / betolt()	Függvény, amely az oldal betöltésekor (a useEffect hookban) hívja meg az API-t (Fetch vagy Axios segítségével).

spa.html

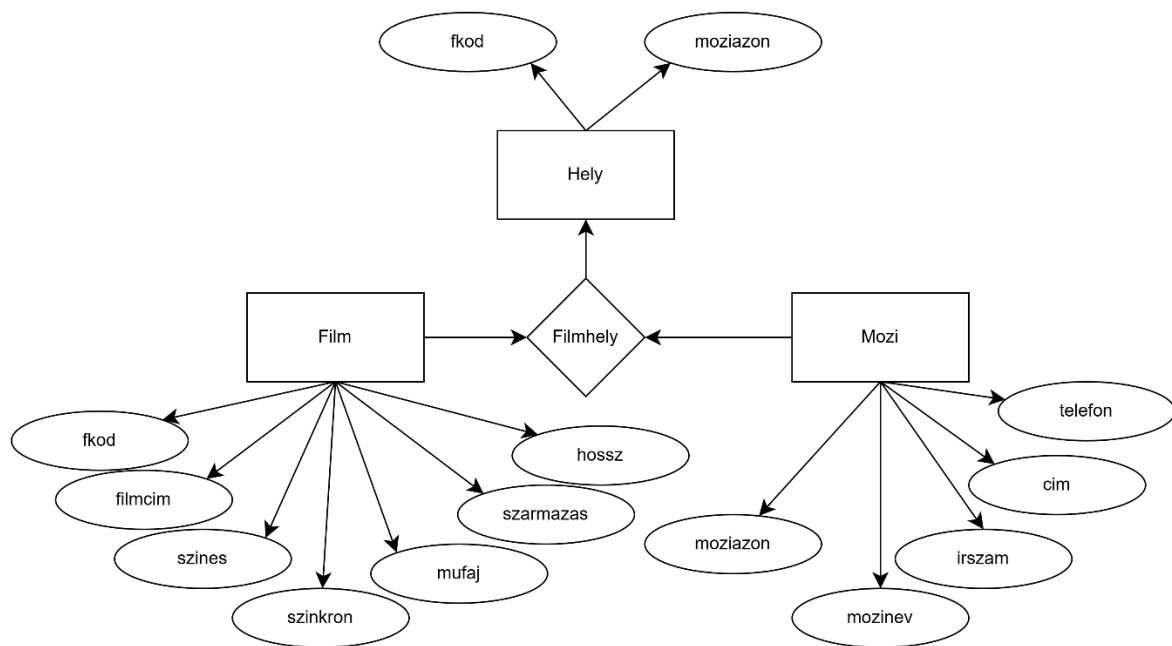
aktivMenu / setAktivMenu	React State (App): Azt vezérli, hogy éppen melyik mini-alkalmazás (Számláló vagy Színváltó) renderelődjön a képernyőn.
szam / setSzam	React State (Szamlalo): A számláló komponens aktuális numerikus értékét tárolja.
szin / setSzin	React State (SzinValto): A színváltó komponens aktuális háttérszínének hexadecimális kódját tárolja.
szinek	Konstans tömb (SzinValto): A választható színek palettáját tartalmazza.

valtoztat()	Függvény (SzinValto): Véletlenszerűen kiválaszt egy új színt a színek tömbből, amely eltér az aktuálistól.
-------------	--

oojs.html

Alakzat	Bázisosztály, amely az általános grafikus tulajdonságokat (x, y koordináták, szín) és a DOM elem létrehozását, valamint a rajzol() metódust tartalmazza.
Teglalap	Az Alakzat osztályból származtatott (öröklött) gyermekosztály. Kiegészül a szélesség és magasság tulajdonságokkal, és felülírja (kiegészíti) a rajzol() metódust.
ujTeglalap()	A gombnyomásra lefutó függvény, amely véletlenszerű paramétereket generál, példányosít egy Teglalap objektumot, és meghívja annak rajzóó függvényét.

Adatbázis



Táblázatok:

Film: fkod, filmcim, színes, szinkron, mufaj, szarmazas, hossz

Hely: fkod, moziazon

Mozi: moziazon, mozinev, irszam, cím, telefon

Kapcsolás:

Kapcsolati adatok:

Host / Ip: localhost

Port: 3306

Adatbázis neve: pmb6zm

Felhasználónév: pmb6zm

Jelszó: Gamf1234

Film tábla:

fkod	filmcim	szines	szinkron	szarmazas	mufaj	hossz
1	Ágnes öcsénk	-1	fel	német	dráma	115
2	Az állomásfőnök	-1	mb	amerikai	dráma	88
3	Álmodozók	-1	fel	angol-francia-olasz	dráma	110
4	Amelie csodálatos élete	-1	mb	francia-német	vígjáték	117
5	Annie Hall	-1	mb	amerikai	vígjáték	89

Hely tábla:

fkod	moziazon
1	8
1	24
2	37
3	21
3	24

Mozi tábla:

moziazon	mozinev	irszam	cim	telefon
1	A38 Hajó	1113	Petőfi híd budai hídfő	4643940
2	Bárka Kikötő a Ráday utcában	1092	Ráday utca	NULL
3	Bem	1024	Margit krt. 5/b.	3168708
4	Budapesti Európai Ifjúsági Központ	1024	Zivatar u. 1-3.	2124080
5	Cervantes Intézet	1000	Vörösmarty u. 32.	NULL
6	Cinema City Csepel Plaza	1212	Rákóczi F. út 154-170.	4258111
7	Cinema City Új Udvar	1036	Bécsi út 38-44.	4378383

Használati dokumentáció

Funkcionális bemutatás (Használati útmutató)

Főoldal (index.html)

A látogatót egy modern, sötét tónusú felület fogadja, amely dinamikusan listázza az adatbázisban szereplő mozikat a Fetch API segítségével.

CRUD Műveletek (JS, React, Fetch, Axios)

Az alkalmazás négy különböző technológiával mutatja be az adatok kezelését: Létrehozás (Create): Új mozi rögzítése név és cím alapján.

Olvasás (Read): Az adatok aszinkron betöltése az oldal újratöltése nélkül.

Törlés (Delete): Meglévő rekordok eltávolítása az azonosító (ID) alapján.

React SPA (spa.html)

Egyoldalas alkalmazás, amely két funkciót tartalmaz:

Számláló: useState hook használatával növelhető, csökkenthető és nullázható érték.

Színváltó: Véletlenszerű színgenerálás a háttérelemhez, elkerülve az egymást követő azonos színeket.

OOJS Grafika (oojs.html)

Objektumorientált megközelítéssel készített grafikus modul:

Alakzat (Bázisosztály): Tárolja a koordinátákat és a színt, kezeli a DOM elem létrehozását.

Téglalap (Származtatott osztály): Az extends és super kulcsszavakkal örököl a bázisosztályból, kiegészítve azt méretekkel és egyedi rajzolási logikával.

Központi API (api.php)

A REST API a HTTP metódusok (GET, POST, DELETE) alapján dönti el a végrehajtandó műveletet.

GET: Lekéri az összes mozit a mozi táblából.

POST: Új mozit szűr be a json_decode segítségével feldolgozott adatokból.

DELETE: Eltávolítja a rekordot a kapott moziazon alapján.

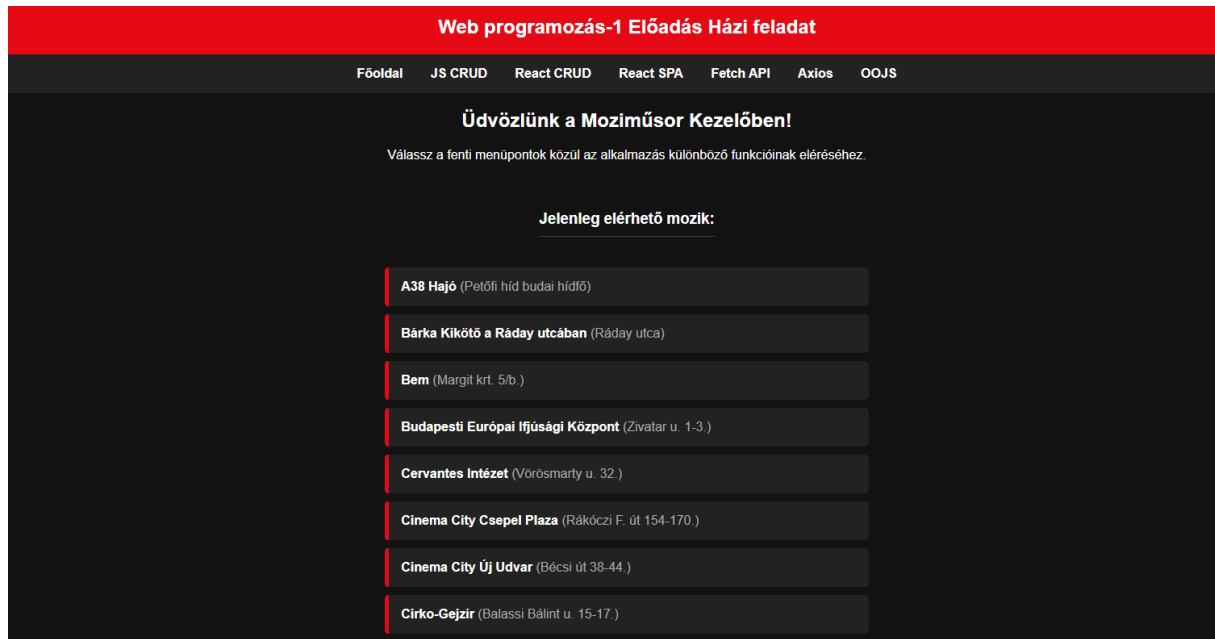
Modern Frontend megoldások

Fetch API: Az async/await szintaxist használjuk a hálózati kérések kezelésére a tisztább kód érdekében.

Axios: Külső könyvtárként segíti a kérések küldését, automatikusan kezelve a JSON konverziót.

React Hooks: A useState biztosítja az adatok szinkronizálását a UI-val, a useEffect pedig az automatikus adatbetöltést az oldal megnyitásakor.

Főoldal



A főoldalon fogad egy kilistázás, ahol láthatjuk melyik moziban vetítik az adott filmet.

Web programozás-1 Előadás Házi feladat

[Főoldal](#)[JS CRUD](#)[React CRUD](#)[React SPA](#)[Fetch API](#)[Axios](#)[OOJS](#)

Mozik Kezelése (JavaScript + API)

Hozzáadás

A38 Hajó (Petőfi híd budai hídfő)	Törlés
Bárka Kikötő a Ráday utcában (Ráday utca)	Törlés
Bem (Margit krt. 5/b.)	Törlés
Budapesti Európai Ifjúsági Központ (Zivatar u. 1-3.)	Törlés
Cervantes Intézet (Vörösmarty u. 32.)	Törlés
Cinema City Csepel Plaza (Rákóczi F. út 154-170.)	Törlés
Cinema City Új Udvar (Bécsi út 38-44.)	Törlés
Cirko-Gejzir (Balassi Bálint u. 15-17.)	Törlés

Itt lehet szerkeszteni azaz hozzáadni és törölni a műsorokat. Javascript programozási nyelvvel megoldva

Web programozás-1 Előadás Házi feladat

[Főoldal](#)[JS CRUD](#)[React CRUD](#)[React SPA](#)[Fetch API](#)[Axios](#)[OOJS](#)

Mozik Kezelése (React CRUD + API)

Hozzáadás

A38 Hajó (Petőfi híd budai hídfő)

Törlés

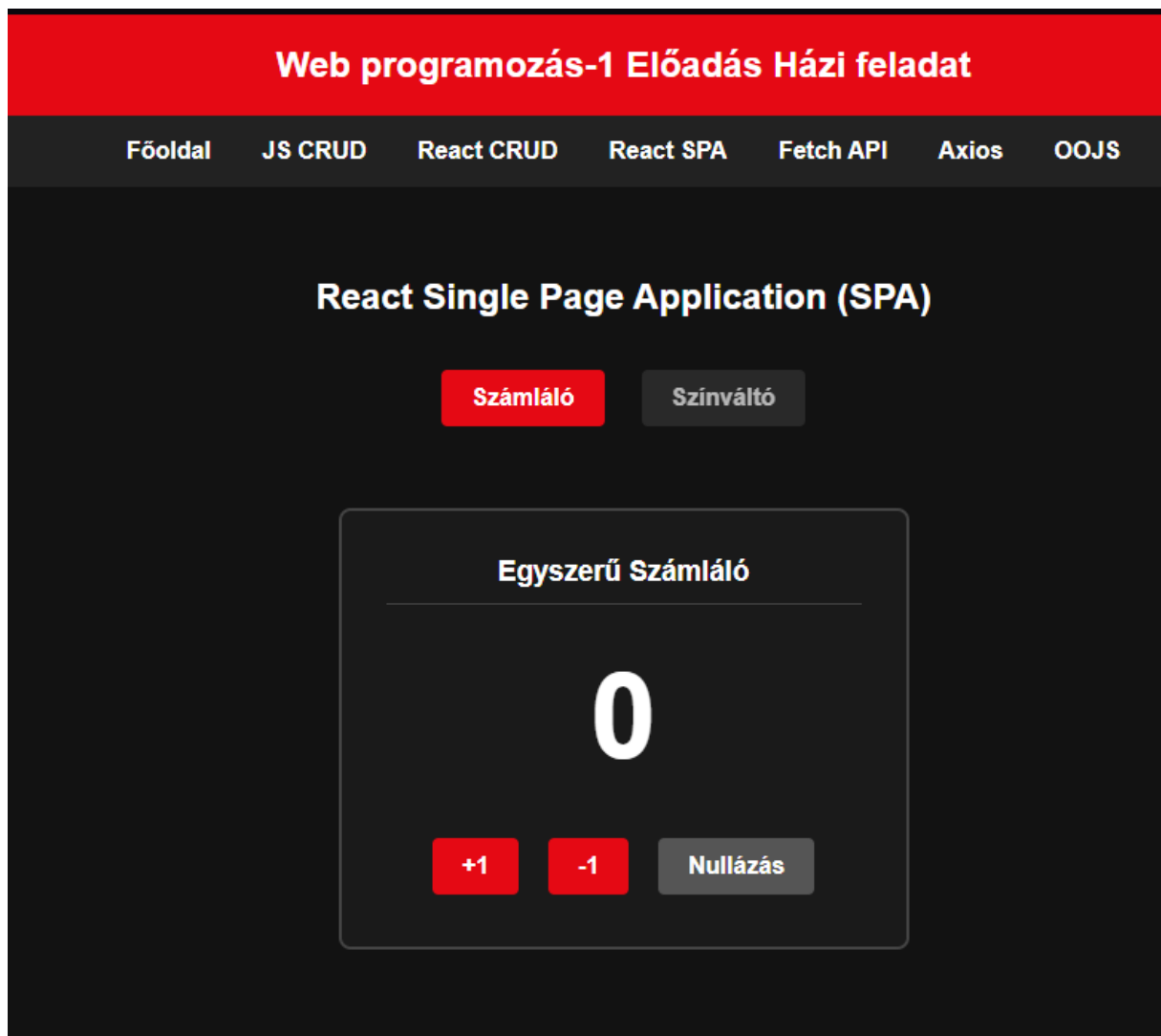
Bárka Kikötő a Ráday utcában (Ráday utca)

Törlés

Bem (Margit krt. 5/b.)

Törlés

Ugyan úgy szerkeszthető csak React-ot használva.



Reactot használva SPA oldal, ami lefordítva „egylapos alkalmazás”.

Az egyszerű számlázásnál hozzáadhatunk és elvehetünk manuálisan számokat a nullázással az alapértékbe a 0-ba teszi a számlálót.

Web programozás-1 Előadás Házi feladat

[Főoldal](#)[JS CRUD](#)[React CRUD](#)[React SPA](#)[Fetch API](#)[Axios](#)[OOJS](#)

React Single Page Application (SPA)

[Számláló](#)[Színváltó](#)

Színváltó Játék

Kattints a gombra a szín megváltoztatásához!

[Szín váltása](#)

A színváltósnál viszont a játék leírás dobozának a háttérszíne változik meg.

Web programozás-1 Előadás Házi feladat

[Főoldal](#)[JS CRUD](#)[React CRUD](#)[React SPA](#)[Fetch API](#)[Axios](#)[OOJS](#)

Mozik Kezelése (Szerver oldali - Fetch API)

Hozzáadás

A38 Hajó (Petőfi híd budai hídfő)

Törlés

Bárka Kikötő a Ráday utcában (Ráday utca)

Törlés

Bem (Margit krt. 5/b.)

Törlés

Az API (Application Programming Interface) magyarul alkalmazásprogramozási felületet, ahol látszólag ugyan az történik mint korábban csak API-n keresztül

AXIOS

Web programozás-1 Előadás Házi feladat

[Főoldal](#)[JS CRUD](#)[React CRUD](#)[React SPA](#)[Fetch API](#)[Axios](#)[OOJS](#)

Mozik Kezelése (Szerver oldali - Axios)

Hozzáadás

A38 Hajó (Petőfi híd budai hídfő)

Törlés

Bárka Kikötő a Ráday utcában (Ráday utca)

Törlés

Bem (Margit krt. 5/b.)

Törlés

Budapesti Európai Ifjúsági Központ (Zivatar u. 1-3.)

Törlés

Az Axios egy nagyon népszerű, harmadik féltől származó JavaScript könyvtár (library), amelynek a feladata, hogy HTTP kéréseket küldjön a weboldalról (kliens oldal) egy szerver felé, és fogadja a válaszokat, ezért ugyan úgy ahogy korábban csak a kivitelezés változott.



Objektum orientált Java script-et objektum generálásával mutattuk be. ahol az „Új téglalap generálásával” véletlenszerű színekkel hozz létre és teszi egymás alá a téglalapokat.

A teljes program kódja magyarázatokkal:

api.php

```
<?php
// 1. ADATBÁZIS KAPCSOLAT BEHÚZÁSA
// A 'require_once' biztosítja, hogy a 'kapcsolat.php' fájl (amiben a $pdo
objektum van)
// betöltődjön, és ha már be lett töltve, nem tölti be újra. Ezzel
csatlakozunk az adatbázishoz.
require_once 'kapcsolat.php';

// 2. VÁLASZ FEJLÉC (HEADER) BEÁLLÍTÁSA
// Megmondjuk a böngészőnek (vagy a frontend kódnak), hogy az adat, amit ez
a fájl
// vissza fog küldeni, az JSON formátumú, és UTF-8 karakterkódolást használ
(így jók lesznek az ékezetek).
header("Content-Type: application/json; charset=UTF-8");

// 3. HTTP KÉRÉS TÍPUSÁNAK LEKÉRDEZÉSE
// A $_SERVER szuperglobális tömbből kiolvassuk, hogy a kliens milyen
típusú kérést küldött.
// Ez lehet 'GET' (olvasás), 'POST' (létrehozás), 'DELETE' (törlés) stb.
$method = $_SERVER['REQUEST_METHOD'];

// 4. ELÁGAZÁS A KÉRÉS TÍPUSA ALAPJÁN
// A switch szerkezet megvizsgálja a $method változó értékét, és a
megfelelő ágot hajtja végre.
switch($method) {

    // --- ADATOK LEKÉRDEZÉSE (OLVASÁS) ---
    case 'GET':
        // Létrehozunk egy SQL lekérdezést, ami lekéri az összes oszlopot
        (*) a 'mozi' táblából.
        $stmt = $pdo->query("SELECT * FROM mozi");
        // A 'fetchAll()' paranccsal az összes megtalált sort kiszedjük egy
        PHP asszociatív tömbbe ($mozik).
        $mozik = $stmt->fetchAll();
        // A json_encode() a PHP tömböt JSON szöveggé alakítja, az 'echo'
        pedig kiírja (visszaküldi a kliensnek).
        echo json_encode($mozik);
        break; // Kilépünk a switch-ből.

    // --- ÚJ ADAT HOZZÁADÁSA (LÉTREHOZÁS) ---
    case 'POST':
        // Mivel a kliens (JavaScript/Axios) JSON formátumban küldi az
        adatokat a kérés "testében",
        // a 'file_get_contents("php://input")' beolvassa ezt a nyers
        szöveget,
        // a 'json_decode()' pedig PHP objektummá alakítja azt.
        $data = json_decode(file_get_contents("php://input"));

        // Biztonsági ellenőrzés: Megnézzük, hogy a kapott objektumban
        létezik-e és nem üres-e
        // a 'mozinev' és a 'cim' tulajdonság.
        if(!empty($data->mozinev) && !empty($data->cim)) {
            // Előkészítjük (prepare) az SQL INSERT utasítást. A kérdőjelek
            (?) helykitöltők.
            // Ezt Prepared Statement-nek hívják, ami megvéd az SQL
            Injection (adatbázis feltörés) ellen.
            $stmt = $pdo->prepare("INSERT INTO mozi (mozinev, cim) VALUES
```

```

(?, ?)");
    // Végrehajtjuk (execute) a lekérdezést, behelyettesítve a
    // kérdőjelek helyére a kapott adatokat.
    $stmt->execute([$data->mozonev, $data->cim]);
    // Visszaküldünk egy JSON üzenetet a kliensnek, hogy tudja,
    // sikeres volt a mentés.
    echo json_encode(["message" => "Sikeres hozzáadás"]);
}
break;

// --- ADAT TÖRLÉSE ---
case 'DELETE':
    // Hasonlóan a POST-hoz, beolvassuk és dekódoljuk a kliens által
    // küldött JSON adatot.
    $data = json_decode(file_get_contents("php://input"));

    // Ellenőrizzük, hogy a kliens elküldte-e a törlendő mozi
    // azonosítóját (moziazon).
    if(!empty($data->moziazon)) {
        // Előkészítjük a törölő SQL utasítást, ahol az azonosító egy
        // paraméter (?).
        $stmt = $pdo->prepare("DELETE FROM mozi WHERE moziazon = ?");
        // Végrehajtjuk a törlést, átadva az ID-t a helykitöltő
        // helyére.
        $stmt->execute([$data->moziazon]);
        // Visszaküldünk egy megerősítő JSON üzenetet a sikeres
        // törlésről.
        echo json_encode(["message" => "Sikeres törlés"]);
    }
    break;
}
?>

```

axios.html

```

<!DOCTYPE html>
<html lang="hu">
<head>
    <meta charset="UTF-8">
    <title>Axios CRUD</title>
    <link rel="stylesheet" href="style.css"> <script
    src="menu.js"></script> <script
    src="https://unpkg.com/react@18/umd/react.development.js"
    crossorigin></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-
    dom.development.js" crossorigin></script>

    <script
    src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

    <script
    src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
</head>
<body>

<main id="root"></main>

<script type="text/babel">
    // A React beépített eszközeinek (Hook-jainak) "kicsomagolása"
    // useState: Változók (állapotok) tárolására szolgál.
    // useEffect: Arra való, hogy az oldal betöltésekor automatikusan
    // lefuttasson kódokat.

```

```

const { useState, useEffect } = React;

// Az API elérési útja (a "pincér" címe)
const API_URL = 'api.php';

// 4. A FŐ REACT KOMPONENS (App)
function App() {
  // ÁLLAPOTOK (State-ek) DEFINIÁLÁSA
  const [mozik, setMozik] = useState([]); // A mozik listáját
  tárolja, alapból egy üres tömb.
  const [nev, setNev] = useState(""); // A "Mozi neve" input mező
  tartalmát tárolja.
  const [cim, setCim] = useState(""); // A "Címe" input mező
  tartalmát tárolja.
  const [hiba, setHiba] = useState(""); // Az esetleges
  hibaüzeneteket tárolja.

  // ADATOK LEKÉRÉSE (READ - GET)
  const betolt = () => {
    // Axios GET kérést küld az api.php-nak
    axios.get(API_URL)
      .then(res => {
        // Ha sikeres a válasz (res), és a válasz adata
        (res.data) egy tömb, akkor eltesszük a mozik állapotba.
        if (Array.isArray(res.data)) {
          setMozik(res.data);
        } else {
          setMozik([]);
        }
      })
      .catch(err => {
        // Ha valami hiba történt (pl. nem fut az adatbázis),
        kiírjuk a hibaüzenetet.
        setHiba("Nem sikerült kapcsolódni az adatbázishoz.
Ellenőrizd a kapcsolat.php-t!");
      });
  };

  // AUTOMATIKUS FUTTATÁS BETÖLTÉSKOR
  // A useEffect az üres tömbbel [] a végén azt jelenti: "Futtasd le
  a betolt() függvényt VEDD ÉS VIDD, de csakis EGYSZER, amikor az oldal
  legelőször betölt!"
  useEffect(() => { betolt(); }, []);

  // ÚJ ADAT FELVITELE (CREATE - POST)
  const hozzaad = () => {
    // Csak akkor csinálunk valamit, ha mindkét mezőbe (nev, cim)
    írtak valamit.
    if(nev && cim) {
      // Axios POST kérést küldünk az api.php-nak. A második
      paraméter maga az adat (JSON formára Axios automatikusan átalakítja).
      axios.post(API_URL, { mozinev: nev, cim: cim })
        .then(() => {
          // Ha sikeresen elmentette a szerver, újra lekérjük
          a listát (betolt()),
          // és kiürítjük az input mezőket (setNev(""),
          setCim("")).

          betolt();
          setNev("");
          setCim("");
        });
    }
  };

```

```

    };

    // ADAT TÖRLÉSE (DELETE)
    const torol = (id) => {
        // Axios DELETE kérést küld az api.php-nak, elküldve a törlendő
        // azonosítót.
        axios.delete(API_URL, { data: { moziazon: id } })
            .then(() => betolt()); // Ha kész, újratölti a listát.
    };

    // 5. A FELÜLET (UI) MEGJELENÍTÉSE
    // A React ezt a HTML-szerű kódot (JSX) rajzolja ki a képernyőre a
    // <main id="root"> belsejébe.
    return (
        <div className="container">
            <h2>Mozik Kezelése (Szerver oldali - Axios)</h2>
            <br/>

            {/* Űrlap rész */}
            <div>
                {/* Az input mezők értéke a State-ekhez (nev, cim) van
                kötve.
                Az onChange figyel, ha gépelsz bele, és azonnal
                frissíti a State-et. */}
                <input type="text" placeholder="Mozi neve" value={nev}
                onChange={ (e) => setNev(e.target.value)} />
                <input type="text" placeholder="Címe" value={cim}
                onChange={ (e) => setCim(e.target.value)} />
                {/* Gombnyomásra meghívja a hozzáad() függvényt */}
                <button onClick={hozzaad}>Hozzáadás</button>
            </div>

            {/* Ha van hiba (a hiba változó nem üres), akkor kirajzol
            egy piros bekezdést a hibaüzenettel. */}
            {hiba && <p style={{color: '#e50914', marginTop: '20px',
            fontWeight: 'bold'}}>{hiba}</p>}

            {/* Feltételes megjelenítés: Ha a mozik listája üres ÉS
            nincs hiba, kiírja, hogy üres az adatbázis. */}
            {mozik.length === 0 && !hiba ? (
                <p style={{marginTop: '40px', color: '#aaa', fontStyle:
                'italic'}}>Az adatbázis jelenleg üres. Kérlek, adj hozzá egy mozit
                fent!</p>
            ) : (
                // Ha vannak mozik, végigmegy a tömbön (map), és
                // mindegyikből csinál egy listaelem-et (<li>)
                <ul>
                    {mozik.map(mozi => (
                        // A key={} nagyon fontos a Reactnak, hogy
                        // tudja, melyik elemet kell frissítenie a listában.
                        <li key={mozi.moziazon}>
                            <div style={{flex: 1}}>
                                <strong>{mozi.mozinev}</strong> <span
                                style={{color: '#aaa'}}>({mozi.cim})</span>
                            </div>
                            {/* A törlés gomb meghívja a torol()
                            függvényt a konkrét mozi azonosítójával */}
                            <button onClick={() =>
                            torol(mozi.moziazon)} style={{marginLeft: '10px', backgroundColor:
                            '#e50914'}}>Törlés</button>

```

```

        </li>
      )}
    </ul>
  )}
</div>
);
}

// 6. AZ ALKALMAZÁS ELINDÍTÁSA
// Megkeressük a <main id="root"> elemet a HTML-ben, és beletesszük
(rendereljük) az App komponenst.
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
</script>
</body>
</html>

```

fetchapi.html

```

<!DOCTYPE html>
<html lang="hu">
<head>
  <meta charset="UTF-8">
  <title>Fetch API CRUD</title>
  <link rel="stylesheet" href="style.css">
  <script src="menu.js"></script>
</head>
<body>

<main class="container">
  <h2>Mozik Kezelése (Szerver oldali - Fetch API)</h2>
  <br>
  <input type="text" id="nev" placeholder="Mozi neve">
  <input type="text" id="cim" placeholder="Címe">

  <button onclick="mentes()">Hozzáadás</button>

  <ul id="lista"></ul>
</main>

<script>
  // Eltároljuk az API útvonalát egy konstansban, hogy ne kelljen mindig
  leírni.
  const API_URL = 'api.php';

  // 1. ADATOK BETÖLTÉSE (READ)
  // Az 'async' (aszinkron) kulcsszó jelzi, hogy ez a függvény olyan
  műveleteket végez (pl. hálózati kérés), ami időbe telik, így a programnak
  "várnia" kell rá.
  async function betoltes() {
    // A 'fetch' alapértelmezetten egy GET kérést indít. Az 'await'
    megállítja a kód futását ezen a soron, amíg meg nem jön a válasz a
    szerverről.
    const res = await fetch(API_URL);

    // A nyers választ JSON formátumból használható JavaScript tömbbé
    alakítjuk. Itt is várni kell (await), amíg az átalakítás lezajlik.
    const adatok = await res.json();

    // Megkeressük a HTML-ben a 'lista' azonosítójú <ul> elemet.
    const lista = document.getElementById('lista');
  }

```



```

        // Kiürítjük a listát, hogy ne duplikálódjanak az elemek, amikor
        újratöltjük.
        lista.innerHTML = '';

        // Végigmegyünk a szerverről kapott tömbön (adatok).
        adatok.forEach(mozi => {
            // Minden egyes mozihoz hozzáadunk (+=) egy új HTML listaelemet
            (<li>) a meglévőkhöz.
            // A ${mozi.mozinev} szintaxis behelyettesíti az adott mozi
            változóit a HTML kódba.
            lista.innerHTML += `<li>
                <div style="flex: 1;"><strong>${mozi.mozinev}</strong>
                <span style="color: #aaa;">(${mozi.cim})</span></div>

                <button onclick="torles(${mozi.moziazon})" style="margin-
                left: 10px; background-color: #e50914;">Törlés</button>
            </li>`;
        });
    }

    // 2. ÚJ ADAT MENTÉSE (CREATE)
    async function mentes() {
        // Kiolvassuk a felhasználó által beírt szövegeket az input
        mezőkből.
        const mozinev = document.getElementById('nev').value;
        const cim = document.getElementById('cim').value;

        // A 'fetch' függvénnyel POST kérést küldünk a szervernek.
        await fetch(API_URL, {
            method: 'POST', // Kérés típusa
            headers: { 'Content-Type': 'application/json' }, // Megmondjuk
            a szervernek, hogy JSON-t küldünk
            body: JSON.stringify({ mozinev, cim }) // A JavaScript
            objektumot JSON szöveggé alakítjuk és betesszük a kérés "testébe".
        });

        // Miután a mentés sikeresen lefutott, újra meghívjuk a betoltes()
        függvényt, hogy frissítse a képernyőn a listát.
        betoltes();
    }

    // 3. ADAT TÖRLÉSE (DELETE)
    // Ez a függvény kap egy 'id' paramétert (a törlendő mozi
    azonosítóját).
    async function torles(id) {
        // DELETE kérést küldünk a 'fetch' segítségével.
        await fetch(API_URL, {
            method: 'DELETE',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ moziazon: id }) // Elküldjük a törlendő
            azonosítót JSON formátumban.
        });

        // Miután a szerver törölte az adatot, újratöltjük a listát a
        képernyőn.
        betoltes();
    }

    // 4. PROGRAM INDÍTÁSA
    // Amikor a böngésző betölti ezt a fájlt és idáig ér a kód olvasásában,

```

```

automatikusan lefuttatja a betoltes() függvényt.
    betoltes();
</script>
</body>
</html>

```

index.html

```

<!DOCTYPE html>
<html lang="hu">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Moziműsor - Web-programozás 1</title>

    <link rel="stylesheet" href="style.css">
    <script src="menu.js"></script>
</head>
<body>

<main class="container">
    <h2>Üdvözlünk a Moziműsor Kezelőben!</h2>
    <p>Válassz a fenti menüpontok közül az alkalmazás különböző funkcióinak
eléréséhez.</p>

    <div style="margin-top: 40px;">
        <h3 style="border-bottom: 1px solid #444; padding-bottom: 10px;
display: inline-block;">Jelenleg elérhető mozik:</h3>

        <ul id="kezdoLista" style="margin-top: 20px;"></ul>
    </div>
</main>

<script>
    // 1. KÉRÉS INDÍTÁSA
    // Lekérjük az adatokat az api.php-től. Mivel nincs megadva 'method',
ez alapértelmezetten GET kérés lesz.
    fetch('api.php')
        // 2. VÁLASZ FELDOLGOZÁSA
        // Amint a szerver válaszol (.then), a nyers HTTP választ JSON
formátumra alakítjuk.
        .then(response => response.json())

        // 3. ADATOK MEGJELENÍTÉSE
        // Amint a JSON átalakítás is kész (.then), megkapjuk a kész tömböt
(adatok).
        .then(adatok => {
            // Megkeressük a HTML-ben a 'kezdoLista' azonosítójú <ul>
elemet.
            const lista = document.getElementById('kezdoLista');

            // ELLENŐRZÉS: Ha a kapott adat üres (nincs még mozi felvéve az
adatbázisba),
            // akkor egy szürke, dőlt betűs tájékoztató szöveget írunk ki.
            if (!adatok || adatok.length === 0) {
                lista.innerHTML = '<li><span style="color: #aaa; font-
style: italic;">Jelenleg nincs adat az adatbázisban. Válassz egy CRUD
menüpontot a hozzáadáshoz!</span></li>';
                return; // Kilépünk a függvényből, nem futtatjuk tovább a
kódot.
            }

```

```

        // Ha vannak adatok, végigmegyünk a tömbön (forEach).
        adatok.forEach(mozi => {
            // Minden egyes mozinál hozzáadunk (+) egy új HTML
            listaelemet (<li>).
            // Figyeld meg: itt nincsen "Törlés" gomb, hiszen ez csak
            egy bemutató lista!
            lista.innerHTML += `
                <li>
                    <div style="flex: 1;">
                        <strong>${mozi.mozinev}</strong> <span
                        style="color: #aaa;">(${mozi.cim})</span>
                    </div>
                </li>`;
        });
    });

    // 4. HIBAKEZELÉS
    // A .catch() ág akkor fut le, ha valami elromlik útközben (pl. nem
    fut a XAMPP Apache/MySQL,
    // vagy hibás az api.php elérési útja).
    .catch(hiba => {
        // A fejlesztőnek kiírjuk a hibát a böngésző konzoljába (F12
        menü).
        console.error("Hiba az adatok lekérésekor:", hiba);
        // A felhasználónak pedig egy szép, piros hibaüzenetet
        jelenítünk meg a lista helyén.
        document.getElementById('kezdoLista').innerHTML = '<li><span
        style="color: #e50914;">Nem sikerült kapcsolódni az
        adatbázishoz.</span></li>';
    });
</script>

</body>
</html>

```

javascript.html

```

<!DOCTYPE html>
<html lang="hu">
<head>
    <meta charset="UTF-8">
    <title>JS Mozik CRUD</title>
    <link rel="stylesheet" href="style.css"> <script
    src="menu.js"></script> </head>
<body>

<main class="container">
    <h2>Mozik Kezelése (JavaScript + API)</h2>
    <br>

    <div>
        <input type="text" id="moziNev" placeholder="Mozi neve">
        <input type="text" id="moziCim" placeholder="Címe">

        <button onclick="hozzaadMozi()">Hozzáadás</button>
    </div>

    <ul id="moziLista"></ul>
</main>

<script>
    // Az API elérési útvonala (szerver címe)

```

```

const API_URL = 'api.php';

// GLOBÁLIS VÁLTOZÓ: Ez fogja tárolni a memóriában az aktuális
mozilistát.
// Így nem kell mindig a HTML-ből kiolvasni az adatokat, ha csak szűrni
vagy módosítani akarjuk a listát.
let mozik = [];

// --- 1. ADATOK BETÖLTÉSE (Hálózat) ---
async function betoltMozik() {
  // A try...catch (próbáld...kapd el) blokk a hibakezelésre szolgál.
  try {
    // Megpróbáljuk lekérni az adatokat a szerverről
    const response = await fetch(API_URL);

    // Ha sikerült, a nyers választ JSON-ból JS tömbbé alakítjuk,
    és eltesszük a 'mozik' globális változóba
    mozik = await response.json();

    // Miután megvannak az adatok a memóriában, meghívjuk a rajzoló
    függvényt
    megjelenitMozik();

  } catch (hiba) {
    // Ha bármi hiba történik (pl. nincs internet, rossz az API
    cím), ide ugrik a kód
    console.error("Hiba:", hiba); // Kiírja a hibát a böngésző
    fejlesztői konzoljába
  }
}

// --- 2. ADATOK MEGJELENÍTÉSE (Rajzolás) ---
// Ennek a függvénynek CSAK az a dolga, hogy a 'mozik' változó
tartalmát kirajzolja a képernyőre.
function megjelenitMozik() {
  // Megkeressük az üres <ul> listát a HTML-ben
  const lista = document.getElementById('moziLista');

  // Kiürítjük, mielőtt újra feltöltjük, hogy ne legyenek duplikációk
  lista.innerHTML = '';

  // Végigmegyünk a memóriában lévő mozik tömbjén
  mozik.forEach((mozi) => {
    // Hozzáadjuk a lista belső HTML-jéhez az új elemeket
    lista.innerHTML += `
      <li>
        <div style="flex: 1;"><strong>${mozi.mozinev}</strong>
        <span style="color: #aaa;">(${mozi.cim})</span></div>
        <button onclick="torolMozi(${mozi.moziazon})"
        style="margin-left: 10px; background-color: #e50914;">Törlés</button>
      </li>`;
  });
}

// --- 3. ÚJ ADAT HOZZÁADÁSA (Mentés) ---
async function hozzaadMozi() {
  // Kiolvassuk az input mezők értékeit
  const mozinev = document.getElementById('moziNev').value;
  const cim = document.getElementById('moziCim').value;

```

```

        // Csak akkor küldünk kérést a szervernek, ha mindkét mező ki van
        töltve
        if(mozinev && cim) {
            // POST kérés küldése az új adatokkal
            await fetch(API_URL, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ mozinev: mozinev, cim: cim })
            });

            // Sikeres mentés után kiürítjük a beviteli mezőket a képernyőn
            document.getElementById('moziNev').value = '';
            document.getElementById('moziCim').value = '';

            // Újra letöltjük a teljes, frissített listát a szerverről
            betoltMozik();
        }
    }

    // --- 4. ADAT TÖRLÉSE (Törlés) ---
    async function torolMozi(id) {
        // DELETE kérés küldése a kiválasztott azonosítóval
        await fetch(API_URL, {
            method: 'DELETE',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ moziazon: id })
        });

        // Törlés után újra lekérjük a megmaradt mozikat a szerverről
        betoltMozik();
    }

    // --- 5. PROGRAM INDÍTÁSA ---
    // Az oldal betöltődésekor rögtön meghívjuk a betöltő függvényt.
    betoltMozik();
</script>
</body>
</html>

```

kapcsolat.php

```

<?php
// --- 1. HIBAKEZELEÉS BEÁLLÍTÁSA ---
// A fejlesztés során fontos tudnunk a hibákról, de éles környezetben (a
// felhasználóknak)
// nem szabad megmutatni őket, mert biztonsági kockázatot jelentenek (pl.
// kiderülhetnek a jelszavak).

// Bekapcsoljuk az összes lehetséges hiba és figyelmeztetés jelentését a
// PHP-ban.
error_reporting(E_ALL);

// display_errors = 0: Letiltjuk, hogy a hibák megjelenjenek a böngésző
// képernyőjén.
ini_set('display_errors', 0);

// log_errors = 1: Engedélyezzük, hogy a hibákat a rendszer egy fájlba
// mentse.
ini_set('log_errors', 1);

// error_log: Megadjuk a fájl pontos helyét és nevét. A ' __DIR__ ' az

```

```

aktuális mappa útvonala,
// így ebbe a mappába fog létrejönni egy 'php_errors.log' nevű fájl. Ide
kerülnek majd a hibaüzenetek.
ini_set('error_log', __DIR__ . '/php_errors.log');

// --- 2. MUNKAMENET (SESSION) ELINDÍTÁSA ---
// A munkamenetek (session-ök) adatok (pl. bejelentkezett felhasználó)
tárolására szolgálnak a lapletöltések között.
// Ez a feltétel megnézi: "Ha még nincs elindítva munkamenet
(PHP_SESSION_NONE), akkor indítsd el!"
// Ezzel elkerülhető a "Session already started" típusú PHP hibaüzenet.
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

// --- 3. ADATBÁZIS KAPCSOLÓDÁSI ADATOK ---
// Itt adjuk meg, hogy hol és milyen jelszóval érhető el a MySQL adatbázis
(XAMPP).
$port = 3306;
$host = 'localhost';
$dbname = 'pmb6zm';
$user = 'pmb6zm';
$pass = 'Gamf1234';
$charset = 'utf8mb4';

// --- 4. DSN (Data Source Name) LÉTREHOZÁSA ---
// Ez egy speciális formátumú "cím", amit a PHP PDO megért. Összefűzzük
benne a fenti változókat.
// Végeredménye kb. ez lesz:
"mysql:host=localhost;port=3306;dbname=mozimisor;charset=utf8mb4"
$dsn = "mysql:host=$host;port=$port;dbname=$dbname;charset=$charset";

// --- 5. PDO BEÁLLÍTÁSOK (OPCIÓK) ---
// Ez egy tömb, ami finomhangolja a kapcsolat működését.
$options = [
    // Ha hiba történik (pl. elírtál egy SQL parancsot), dobjon egy
    "Kivételt" (Exception), amit el tudunk kapni.
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,

    // A lekérdezések eredményét alapértelmezetten asszociatív tömbként
    adja vissza (pl. $sor['mazinev'] formában).
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,

    // Kikapcsoljuk az emulált előkészített utasításokat. Ez arra
    kényszeríti a PHP-t,
    // hogy a valódi, biztonságos MySQL Prepared Statements funkciót
    használja (véd az SQL Injection ellen).
    PDO::ATTR_EMULATE_PREPARES => false,
];

// --- 6. A KAPCSOLAT FELÉPÍTÉSE (TRY-CATCH BLOKK) ---
// A try blokkban "megpróbáljuk" lefuttatni a kockázatos kódot (a
csatlakozást).
try {
    // Létrehozunk egy új PDO (PHP Data Objects) objektumot a korábban

```

```

megadott adatokkal.
    // Ezt a '$pdo' változót fogja majd használni az 'api.php' a
lekérdezésekhez!
    $pdo = new PDO($dsn, $user, $pass, $options);

} catch (\PDOException $e) {
    // Ha a csatlakozás sikertelen (pl. nem fut a XAMPP MySQL modulja, vagy
rossz a jelszó),
    // a program nem omlik össze csúnyán, hanem "elkapja" a hibát (catch).

    // 1. A valódi, technikai hibaüzenetet beírjuk a log fájlba
(PHP_ERRORS_LOG), hogy mi, fejlesztők lássuk.
    error_log("Adatbázis csatlakozási hiba: " . $e->getMessage());

    // 2. A program futását megállítjuk (die), és a látogatónak csak egy
barátságos üzenetet mutatunk.
    die("Szerverhiba történt. Kérjük, nézze meg a hibanaplót.");
}
?>

```

menu.js

```

// --- 1. A FEJLÉC ÉS A MENÜSÁV DEFINIÁLÁSA ---
// A 'const' kulcsszóval létrehozunk egy állandó változót (headerEsNav).
// A visszafelé dőlő idézőjelek (backtick: ` `) egy úgynevezett "Template
Literal"-t jelölnek.
// Ez nagyon hasznos, mert engedi, hogy több sorba tördelve, kényelmesen
írjunk HTML kódot a JavaScripten belül.
const headerEsNav = `
<header>
    <h1>Web programozás-1 Előadás Házi feladat</h1>
</header>
<nav>
    <a href="index.html">Főoldal</a>
    <a href="javascript.html">JS CRUD</a>
    <a href="React.html">React CRUD</a>
    <a href="spa.html">React SPA</a>
    <a href="fetchapi.html">Fetch API</a>
    <a href="axios.html">Axios</a>
    <a href="oojs.html">OOJS</a>
</nav>
`;

// --- 2. A LÁBLÉC DEFINIÁLÁSA ---
// Hasonlóan az előzőhöz, itt a lábléc (footer) HTML kódját mentjük el egy
változóba.
const footer = `
<footer>
    Készítette: Palotai Tamás (PMB6ZM) és Horvát Evelin (AJ4B6Q)
</footer>
`;

// --- 3. AZ ELEMEEK BEILLESZTÉSE AZ OLDALBA ---
// A 'document.addEventListener' figyel a böngésző eseményeit.
// A "DOMContentLoaded" egy nagyon fontos esemény: akkor fut le, amikor a
böngésző
// már beolvasta a HTML fájlt, de még nem feltétlenül töltötte be a képeket
és a stílusokat.
// Ilyenkor már biztonságosan tudjuk módosítani az oldal szerkezetét.
document.addEventListener("DOMContentLoaded", () => {

```

```

    // A 'document.body' az aktuális weblap <body> címkéjét jelenti.
    // Az 'insertAdjacentHTML' egy olyan függvény, amivel nyers HTML
    szöveget tudunk "beragasztani" az oldalba.

    // Az "afterbegin" paraméter azt jelenti: "Tedd a <body>
    legeslegelejére, mint az első elemet!"
    // Ezzel a fejléc és a menü mindig az oldal legtetején fog megjelenni.
    document.body.insertAdjacentHTML("afterbegin", headerEsNav);

    // A "beforeend" paraméter azt jelenti: "Tedd a <body> legeslegvégére,
    mielőtt lezárulna a </body> címke!"
    // Ezzel biztosítjuk, hogy a lábléc mindig a tartalom alá, az oldal
    legaljára kerüljön.
    document.body.insertAdjacentHTML("beforeend", footer);
});

```

oojs.html

```

<!DOCTYPE html>
<html lang="hu">
<head>
  <meta charset="UTF-8">
  <title>OOJS Grafika</title>
  <link rel="stylesheet" href="style.css">
  <script src="menu.js"></script>
</head>
<body>

<main class="container">
  <h2>Objektumorientált JS (OOJS)</h2>
  <br>
  <button onclick="ujTeglalap()">Új Téglalap generálása</button>
</main>

<script>
  // --- 1. A BÁZISOSZTÁLY LÉTREHOZÁSA (Szülő osztály) ---
  // A 'class' kulcsszóval definiálunk egy sablont (tervrajzot).
  class Alakzat {
    // A 'constructor' egy speciális függvény, ami mindig automatikusan
    lefut,
    // amikor létrehozunk egy új alakzatot ebből a sablonból.
    constructor(x, y, szin) {
      this.x = x;      // Vízszintes pozíció elmentése az adott objektumba
      (this)
      this.y = y;      // Függőleges pozíció elmentése
      this.szin = szin; // Szín elmentése

      // Létrehozunk a memóriában egy új <div> HTML elemet, de még nem
      tesszük ki a képernyőre.
      this.elem = document.createElement("div");
      // Adunk neki egy CSS osztályt, hogy a style.css-ből megkapja az alap
      kinézetet (pl. 'absolute' pozíció).
      this.elem.className = "alakzat";
    }

    // Ez egy metódus (az osztály saját függvénye), ami megjeleníti az
    alakzatot.
    rajzol() {
      // Beállítjuk a CSS 'left' és 'top' tulajdonságokat a koordináták
      alapján.
      // (A + "px" azért kell, mert a CSS pixeleken várja az értéket).

```



```

        this.elem.style.left = this.x + "px";
        this.elem.style.top = this.y + "px";
        // Beállítjuk a háttérszínt.
        this.elem.style.backgroundColor = this.szin;

        // Végül "hozzáragasztjuk" a memóriában lévő <div> elemet a HTML-ben
        // található <main> részhez, így megjelenik a képernyőn.
        document.querySelector("main").appendChild(this.elem);
    }
}

// --- 2. ÖRÖKLŐDÉS (Gyermek osztály) ---
// A 'Teglalap' osztály az 'Alakzat' osztályból származik (extends).
// Megörökli annak minden tulajdonságát (x, y, szín) és képességét
// (rajzol).
class Teglalap extends Alakzat {

    // A téglalapnak már szélessége és magassága is van, így ezeket is
    // bekérjük.
    constructor(x, y, szin, szelesseg, magassag) {
        // A 'super()' meghívja a szülő (Alakzat) constructorát.
        // Olyan, mintha azt mondanánk: "Szülő, légy szíves állítsd be az x-
        // et, y-t és a színt, mert ahhoz te értesz!"
        super(x, y, szin);

        // A szélességet és magasságot már a téglalap saját maga kezeli.
        this.szelesseg = szelesseg;
        this.magassag = magassag;
    }

    // A téglalap felülírja a szülő 'rajzol' metódusát (ezt hívják
    // polimorfizmusnak/többalakúságnak).
    rajzol() {
        // Beállítja a specifikus CSS tulajdonságokat (szélesség, magasság).
        this.elem.style.width = this.szelesseg + "px";
        this.elem.style.height = this.magassag + "px";

        // Majd meghívja a szülő 'rajzol()' metódusát ('super.rajzol()'),
        // ami elvégzi a maradék munkát (pozicionálás, színezés és képernyőre
        // rakás).
        super.rajzol();
    }
}

// --- 3. OBJEKTUMOK LÉTREHOZÁSA (Példányosítás) ---
// Ez a függvény fut le minden alkalommal, amikor rákattintasz a gombra.
function ujTeglalap() {
    // 1. Véletlenszerű X koordináta generálása (50 és 550 pixel között)
    const x = Math.floor(Math.random() * 500) + 50;
    // 2. Véletlenszerű Y koordináta generálása (300 és 600 pixel között,
    // hogy a gomb alá kerüljön)
    const y = Math.floor(Math.random() * 300) + 300;

    // 3. Választható színek tömbje
    const szinek = ["red", "blue", "yellow", "green"];
    // Véletlenszerű szín kiválasztása a tömbből
    const randomSzin = szinek[Math.floor(Math.random() * szinek.length)];

    // LÉTREHOZÁS: Példányosítunk egy új Téglalapot a véletlen adatokkal,
    // FIX 100px szélességgel és 50px magassággal.
    // A 'new' kulcsszó hozza létre a konkrét objektumot a sablonból.

```

```

    const tegla = new Teglalap(x, y, randomSzin, 100, 50);

    // MEGJELENÍTÉS: Utasítjuk a frissen létrejött téglalapot, hogy
    rajzolja ki magát.
    tegla.rajzol();
  }
</script>
</body>
</html>

```

React.html

```

<!DOCTYPE html>
<html lang="hu">
<head>
  <meta charset="UTF-8">
  <title>React CRUD</title>
  <link rel="stylesheet" href="style.css"> <script
src="menu.js"></script> <script
src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js" crossorigin></script>

  <script
src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>

<main id="root"></main>

<script type="text/babel">

  // Kibontjuk a React legfontosabb Hook-jait (eszközeit).
  // useState: Változók (állapotok) figyelése.
  // useEffect: Oldalbetöltéskor lefutó kódok kezelése.
  const { useState, useEffect } = React;

  // Az API elérési útvonala
  const API_URL = 'api.php';

  // --- 4. A FŐ KOMPONENS (App) LÉTREHOZÁSA ---
  function App() {

    // --- ÁLLAPOTOK (STATE-EK) DEFINIÁLÁSA ---
    // A React ezeket a változókat figyeli. Ha bármelyik megváltozik,
    azonnal újrarajzolja a képernyő érintett részét!
    const [mozik, setMozik] = useState([]); // A mozik listája,
    kezdetben egy üres tömb.
    const [nev, setNev] = useState(""); // A név beviteli mező
    értéke.
    const [cim, setCim] = useState(""); // A cím beviteli mező
    értéke.

    // --- ADATOK BETÖLTÉSE (Fetch API-val) ---
    const betoltMozik = () => {
      fetch(API_URL) // GET kérést indítunk a szerver felé
        .then(res => res.json()) // A választ JSON formátumról
        JavaScript tömbbé alakítjuk
        .then(data => {
          // Ha a kapott adat tényleg egy tömb (biztonsági

```

```

ellenőrzés), akkor betesszük a 'mozik' állapotba.
        if (Array.isArray(data)) setMozik(data);
    });

    };

    // --- AUTOMATIKUS FUTTATÁS ---
    // Az üres tömb [] miatt ez a függvény pontosan egyszer fut le:
    amikor a felület először betölt.
    useEffect(() => { betoltMozik(); }, []);

    // --- ÚJ ADAT HOZZÁADÁSA ---
    const hozzaad = () => {
        // Csak akkor indul el a kérés, ha a felhasználó írt valamit
        // mindkét mezőbe (nev és cim).
        if (nev && cim) {
            // POST kérést küldünk a fetch segítségével
            fetch(API_URL, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ mozinev: nev, cim: cim }) //
                Elküldjük a state-ben lévő adatokat
            }).then(() => {
                // Ha a szerver elmentette, frissítjük a képernyőt:
                betoltMozik(); // Újratöltjük a listát
                setNev(""); // Kiürítjük a név input mezőt
                setCim(""); // Kiürítjük a cím input mezőt
            });
        }
    };

    // --- ADAT TÖRLÉSE ---
    const torol = (id) => {
        // DELETE kérést küldünk, benne a törlendő azonosítóval.
        fetch(API_URL, {
            method: 'DELETE',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ moziazon: id })
        }).then(() => betoltMozik()); // Törlés után rögtön újratöltjük
        a listát.
    };

    // --- 5. A FELÜLET (JSX) MEGJELENÍTÉSE ---
    return (
        <div className="container">
            <h2>Mozik Kezelése (React CRUD + API)</h2>
            <br/>

            { /* Űrlap rész */ }
            <div>
                { /* Az input mezők értéke hozzá van láncolva a React
                State-hez.
                Az onChange figyel a gépelést (e.target.value az
                épp beírt szöveg),
                és azonnal frissíti a 'nev' vagy 'cim' változót.
                */ }

                <input type="text" placeholder="Mozi neve" value={nev}
                onChange={ (e) => setNev(e.target.value)} />
                <input type="text" placeholder="Címe" value={cim}
                onChange={ (e) => setCim(e.target.value)} />
                <button onClick={hozzaad}>Hozzáadás</button>
            </div>

```

```

        {/* A mozik listájának megjelenítése */}
        <ul>
            {/* A .map() végigmegy a 'mozik' tömbön, és minden
elemből generál egy <li> listakezdeményt. */}
            {mozik.map(mozi => (
                // A 'key' megadása kötelező Reactban listák
esetén, ebből tudja követni, melyik elem melyik!
                <li key={mozi.moziazon}>
                    <div style={{flex: 1}}>
                        <strong>{mozi.mozinev}</strong> <span
style={{color: '#aaa'}}>({mozi.cim})</span>
                    </div>
                    {/* A Törlés gomb kattintáskor meghívja a
torol() függvényt a konkrét mozi azonosítójával */}
                    <button onClick={() => torol(mozi.moziazon)}
style={{marginLeft: '10px', backgroundColor: '#e50914'}}>Törlés</button>
                </li>
            ))}
        </ul>
    </div>
);
}

// --- 6. AZ ALKALMAZÁS ELINDÍTÁSA ---
// Megfogjuk a HTML-ben lévő <main id="root"> elemet, és kirajzoljuk
bele a fent elkészített App komponensst.
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
</script>
</body>
</html>

```

spa.html

```

<!DOCTYPE html>
<html lang="hu">
<head>
    <meta charset="UTF-8">
    <title>React SPA</title>
    <link rel="stylesheet" href="style.css"> <script src="menu.js"></script>
    <script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>

    <main id="root"></main>

    <script type="text/babel">

        // Csak a 'useState' Hook-ot kérjük el, mert itt most nem töltünk le
adatokat a szerverről (nincs useEffect).
        const { useState } = React;

        // --- 1. KOMPONENS: SZÁMLÁLÓ ---
        // Ez egy önálló, saját logikával rendelkező mini-alkalmazás.
        function Szamlalo() {
            // Állapot (State) definiálása a számnak, kezdőértéke: 0.

```

```

    const [szam, setSzam] = useState(0);

    return (
      <div className="app-container" style={{ textAlign: 'center',
minWidth: '320px' }}>
        <h3 style={{ borderBottom: '1px solid #444', paddingBottom:
'10px', marginTop: '0' }}>Egyszerű Számláló</h3>

        {/* A 'szam' változó értékét írjuk ki hatalmas betűkkel */}
        <div style={{ fontSize: '5rem', fontWeight: 'bold', margin:
'30px 0', color: '#ffffff' }}>{szam}</div>

        {/* Gombok a számításokhoz */}
        <div style={{ display: 'flex', justifyContent: 'center', gap:
'10px' }}>

          {/* Gombnyomásra a jelenlegi számhoz hozzáad 1-et */}
          <button onClick={() => setSzam(szam + 1)}>+1</button>

          {/* Gombnyomásra a jelenlegi számból kivon 1-et */}
          <button onClick={() => setSzam(szam - 1)}>-1</button>

          {/* Gombnyomásra visszaállítja a számot 0-ra */}
          <button onClick={() => setSzam(0)} style={{
backgroundColor: '#555' }}>Nullázás</button>
        </div>
      </div>
    );
  }

  // --- 2. KOMPONENS: SZÍNVÁLTÓ ---
  // Ez egy másik önálló mini-alkalmazás.
  function SzinValto() {
    // Állapot definiálása a háttérszínnek, egy alapértelmezett kék színnel
    (Hex kód).
    const [szin, setSzin] = useState("#00509E");

    // Elérhető színek palettája.
    const szinek = ["#00509E", "#B20710", "#2E7D32", "#E65100", "#6A1B9A",
"#00838F"];

    // Függvény a színváltáshoz
    const valtoztat = () => {
      // 1. Kiszűrjük a jelenlegi színt a tömbből, hogy biztosan valami
      újat kapjunk (ne váltson kékből kékre).
      const elerhetőSzinek = szinek.filter(sz => sz !== szin);
      // 2. Véletlenszerűen kiválasztunk egyet a megmaradt színek közül.
      const veletlen = elerhetőSzinek[Math.floor(Math.random() *
elerhetőSzinek.length)];
      // 3. Beállítjuk az új színt a State-be.
      setSzin(veletlen);
    };

    return (
      // Ennek a doboznak a háttérszíne a 'szin' állapothoz van
      kötve. Ha a State változik, a háttér is azonnal!
      <div className="app-container" style={{
        backgroundColor: szin, color: 'white', transition:
'background-color 0.6s ease',
        textAlign: 'center', minWidth: '320px', minHeight: '220px',
        display: 'flex', flexDirection: 'column', justifyContent:
'center', alignItems: 'center',

```

```

        border: 'none', boxShadow: '0 4px 15px rgba(0,0,0,0.5)'
    })>
        <h3 style={{ textShadow: '1px 1px 3px rgba(0,0,0,0.6)',
margin: '0 0 10px 0' }}>Színváltó Játék</h3>
        <p style={{ textShadow: '1px 1px 3px rgba(0,0,0,0.6)',
marginBottom: '20px' }}>Kattints a gombra a szín megváltoztatásához!</p>

        {/* A gomb megnyomása aktiválja a valtoztat() függvényt */}
        <button onClick={valtoztat} style={{ backgroundColor:
'rgba(0,0,0,0.4)', border: '1px solid rgba(255,255,255,0.5)' }}>Szín
váltása</button>
    </div>
    );
}

// --- 3. FŐ KOMPONENS: App (Az "Irányító") ---
function App() {
    // Ez a State határozza meg, hogy éppen melyik mini-alkalmazást lássuk.
    Alapból a "szamlalo"-t.
    const [aktivMenu, setAktivMenu] = useState("szamlalo");

    return (
        <div className="container">
            <h2>React Single Page Application (SPA)</h2>

            {/* BELSŐ NAVIGÁCIÓ (Menü a Reacton belül) */}
            <div style={{ display: 'flex', justifyContent: 'center', gap:
'15px', margin: '30px 0' }}>

                {/* Ha rákattintanak, a State "szamlalo"-ra vált. A gomb
színe dinamikusan változik, ha ő az aktív. */}
                <button onClick={() => setAktivMenu("szamlalo")}
                    style={{ backgroundColor: aktivMenu === "szamlalo"
? "#e50914" : "#333", border: "none", opacity: aktivMenu === "szamlalo" ?
"1" : "0.7" }}>
                    Számláló
                </button>

                {/* Ha rákattintanak, a State "szinvalto"-ra vált. */}
                <button onClick={() => setAktivMenu("szinvalto")}
                    style={{ backgroundColor: aktivMenu === "szinvalto"
? "#e50914" : "#333", border: "none", opacity: aktivMenu === "szinvalto" ?
"1" : "0.7" }}>
                    Színváltó
                </button>
            </div>

            {/* FELTÉTELES MEGJELENÍTÉS (A Varázslat itt történik) */}
            <div style={{ display: 'flex', justifyContent: 'center' }}>
                {/* Ez egy "Ternary operator" (rövidített if-else szabály).
Logika: Ha az aktivMenu értéke "szamlalo", akkor rajzold ki a
<Szamlalo /> komponensst.
Ha NEM (kettőspont utáni rész), akkor rajzold ki a <SzinValto
/> komponensst!
*/}
                {aktivMenu === "szamlalo" ? <Szamlalo /> : <SzinValto />}
            </div>
        </div>
    );
}

```

```
// --- 4. AZ ALKALMAZÁS ELINDÍTÁSA ---
// A fő App komponenst betesszük a HTML-be.
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
</script>
</body>
</html>
```

style.css

```
/* --- ÁLTALÁNOS BEÁLLÍTÁSOK --- */
body {
  /* Az egész weblap alapértelmezett betűtípusa Arial lesz. Ha nincs,
  akkor valamilyen talpatlan (sans-serif) betűtípus. */
  font-family: Arial, sans-serif;
  background-color: #121212; /* Nagyon sötét, szinte fekete háttérszín */
  color: #ffffff; /* Fehér szövegszín a jó kontraszt érdekében */
  margin: 0; /* Eltávolítjuk a böngésző alapértelmezett
  külső margóját, így az oldal a szélekig ér */

  /* FLEXBOX ELRENDEZÉS AZ OLDALON: Ez egy modern trükk a lábléc (footer)
  lentre igazításához. */
  display: flex;
  flex-direction: column; /* Az elemeket (header, nav, main, footer)
  oszlopszerűen, egymás alá rendezi */

  /* A böngészőablak magasságának legalább 100%-át (100 Viewport Height)
  kitölti.
  Ez garantálja, hogy ha kevés a tartalom, a lábléc akkor sem ugrik
  fel az oldal közepére! */
  min-height: 100vh;
}

/* --- FEJLÉC (HEADER) --- */
header {
  background-color: #e50914; /* Jellegzetes "Netflix piros" háttérszín */
  padding: 20px; /* Belső margó: 20 pixel üres tér a szöveg
  körül */
  text-align: center; /* Középre igazítja a szöveget */
}

header h1 {
  margin: 0; /* Ne legyen extra üres hely a címsor
  felett/alatt */
  font-size: 24px; /* Betűméret beállítása */
  color: #ffffff;
}

/* --- MENÜSÁV (NAV) --- */
nav {
  background-color: #222222; /* Sötétszürke háttér a menünek */
  padding: 15px;
  text-align: center;
}

nav a {
  color: white; /* A linkek színe fehér (alapból kék lenne
  és aláhúzott) */
  text-decoration: none; /* Eltüntetjük az aláhúzást a linkekről */
  margin: 0 15px; /* A linkek között bal-jobb irányban 15-15
```

```

pixel szünetet tart */
    font-weight: bold;          /* Félkövér betűtípus */
    font-size: 16px;

    /* Kényelmi funkció: Ha megváltozik a színe (pl. ráviszed az egeret),
       azt ne hirtelen tegye, hanem egy lágy, 0.3 másodperces átmenettel.
    */
    transition: color 0.3s;
}

/* A ':hover' azt jelenti, mi történjen, ha a felhasználó ráviszi az egeret
a linkre. */
nav a:hover {
    color: #e50914; /* Rámutatáskor a szöveg színe pirosra vált */
}

/* --- FŐ TARTALOM (MAIN) --- */
main {
    /* Ez a Flexbox varázslata: a 'flex: 1' megmondja a <main> elemnek,
    hogy
        "nyújtózz ki", és töltsd ki az összes maradék helyet a fejléc és a
    lábléc között! */
    flex: 1;
    padding: 40px;
    text-align: center;
}

main h2 {
    margin-top: 0; /* Eltünteti a címsor feletti extra margót */
}

/* --- LÁBLÉC (FOOTER) --- */
footer {
    background-color: #222222;
    text-align: center;
    padding: 15px;

    /* Mivel a <body> flex elrendezésű, a margin-top: auto szó szerint
       "letolja" a lábléct a képernyő legalsó pontjára. */
    margin-top: auto;

    border-top: 2px solid #e50914; /* Egy 2 pixel vastag piros vonal a
    lábléc tetejére (dizájnelem) */
    font-size: 14px;
}

/* --- ALOLDALAK (CRUD, SPA) STÍLUSAI --- */

/* Ezt az osztályt használd a fő tartalmi blokkok körül, hogy ne nyúljanak
szét a végtelenségig egy nagy monitoron. */
.container {
    max-width: 800px; /* Az elem sosem lesz szélesebb 800 pixelnél */

    /* A 'margin: 0 auto' egy klasszikus CSS trükk a középre igazításra:
       fent/lent 0 margó, bal/jobbs oldalon pedig automatikusan elosztja a
    maradék helyet. */
    margin: 0 auto;
    padding: 20px;
    text-align: center;
}

```



```

/* Hivatkozások az aloldalakon */
a {
    color: #e50914;
    text-decoration: none;
}

a:hover {
    text-decoration: underline; /* Rámutatáskor aláhúzza a szöveget */
}

/* --- Űrlap elemek (Inputok és Gombok) --- */
input[type="text"], input[type="number"] {
    padding: 10px; /* Kényelmes belső tér a gépeléshez */
    margin: 5px;
    border: 1px solid #444; /* Halvány szürke keret */
    border-radius: 4px; /* Enyhén lekerekített sarkok a modern
hatásért */
    background-color: #333; /* Sötét beviteli mező */
    color: white; /* Fehér betűvel írunk bele */
    font-size: 16px;
}

button {
    padding: 10px 20px;
    margin: 5px;
    background-color: #e50914; /* Piros gomb */
    color: white;
    border: none; /* Eltűntetjük a böngésző alap gombkeretét
*/
    border-radius: 4px; /* Lekerekített sarkok */

    /* Amikor az egér a gomb fölé ér, a mutató átalakul egy kis kézzé
(kattinthatóságot jelezve) */
    cursor: pointer;
    font-weight: bold;
    font-size: 16px;
    transition: background-color 0.3s; /* Lágyszínátmenet a hover
effekthez */
}

/* Gomb hover (rámutatás) állapot */
button:hover {
    background-color: #b20710; /* Egy picit sötétebb piros lesz, így
vizuális visszajelzést ad a gomb */
}

/* --- Listák (Mozik listázásához) --- */
ul {
    list-style-type: none; /* Eltűntetjük a hagyományos lista-pöttyöket
*/
    padding: 0;
    max-width: 600px; /* A lista ne legyen szélesebb 600 pixelnél
*/
    margin: 20px auto; /* Középre igazítja a listát az oldalon */
}

li {
    background-color: #222; /* Sötétszürke doboz a listaelemeknek */
    margin: 10px 0; /* A lista elemei között 10 pixel függőleges
távolság van */
    padding: 15px;

```

```

border-radius: 5px;

/* Dizájn elem: egy vastagabb piros csík a doboz bal szélén */
border-left: 5px solid #e50914;

/* Flexbox elrendezés a listaelemen belül is! */
display: flex;

/* A mozi nevét kitolja teljesen balra, a Törlés gombot pedig teljesen
jobbra */
justify-content: space-between;

/* Függőlegesen középre igazítja a szöveget és a gombot egy vonalba */
align-items: center;
text-align: left;
}

/* --- React SPA alkalmazások konténerei --- */
.app-container {
  margin-top: 20px;
  padding: 30px;
  border: 2px solid #444;      /* Vastagabb szürke keret */
  border-radius: 8px;        /* Jobban lekerekített sarkok */

  /* Hogy a doboz csak akkora legyen, amekkora a tartalma (ne nyúljon ki
az egész képernyőre) */
  display: inline-block;
  background-color: #1a1a1a;
}

```